

Technique	Relationship	Signature
Decrease & Conquer (U5)	solve ONE smaller instance, then extend	Topological sort, insertion sort, binary search
Divide & Conquer (U6)	split into SEVERAL subproblems, solve, combine	Quicksort/Hoare, mergesort
Transform & Conquer (U7)	transform to better representation, then solve	Heapify, presorting
Space-Time (U8)	precompute/store for fast queries	Counting sort, Horspool, hashing
Dynamic Programming (U9)	OVERLAPPING subproblems solved once into a table	Coin-row, change, 0/1 knapsack, Floyd, Warshall
Greedy (U10)	sequence of locally-best, unalterable choices	Prim, Kruskal, Dijkstra, Huffman
Backtracking / B&B	state-space tree with bounds & pruning	Assignment problem

DP vs D&C: both split — but D&C subproblems are **independent**; DP exists **because they overlap**. · **Greedy vs DP:** greedy optimizes the **next step** (fast, needs greedy-choice property); DP optimizes the **whole path** via a table (always correct where it applies).

1. Topological Sort · Decrease&Conquer

getting dressed: socks before shoes
Linear order of a **DAG** so every edge $u \rightarrow v$ has u before v .
Source-removal: ① find a source (in-degree 0) ② output & remove it (delete its out-edges) ③ neighbors' in-degrees drop, new sources appear ④ repeat.
Vertices left but no source $\Rightarrow$ **cycle**.
 $\Theta(V+E)$

2. Hoare Partition · Divide&Conquer

line up around a reference person
Engine of quicksort: pivot $p=A[l]$, $\leq p$ left / $\geq p$ right; pivot lands final.
 $i=l$, $j=r+1$. Scan i **right** to first $A[i] \geq p$; j **left** to first $A[j] \leq p$. If $i < j$ swap & repeat.
When $i \geq j$, swap $A[l], A[j] \rightarrow j = \text{split}$.
 $\Theta(n \log n)$ avg · $\Theta(n^2)$ worst

3. Bottom-up Heapify · Transform&Conquer

corporate reorg from the bottom
Max-heap in array: $\text{parent} \geq \text{children}$, children of i at $2i+1$, $2i+2$.
For $i = \lfloor n/2 \rfloor - 1$ down to 0: **sift down** — compare with **larger** child, swap if smaller, continue from new spot; stop at leaf or $\geq$ both children.
build $\Theta(n)$ · heapsort $\Theta(n \log n)$

4. Kruskal MST · Greedy

cheapest roads, skip loops
Min-weight acyclic connected subgraph, $|V|-1$ edges.
① sort edges $\uparrow$ weight ② add each edge if it makes **no cycle** (union-find) ③ stop at $|V|-1$ edges.
 $\Theta(E \log E)$

5. Prim MST · Greedy

a spreading vine from one seed
Grow **one** tree; add cheapest edge crossing tree $\rightarrow$ outside.
① start any vertex ② pick min-weight edge from tree to an outside vertex ③ add edge+vertex ④ until $|V|-1$ edges. **Prim**=one tree/cheapest fringe;
Kruskal=global cheapest/fragments.
 $\Theta(E \log V)$

6. Comparison Counting Sort · Space-Time

how many did you beat?
Count how many elements are smaller than each; that count is its index.
For each pair $i < j$: $A[i] < A[j] \Rightarrow \text{Count}[j]++$ else $\text{Count}[i]++$. Place $A[i]$ at position $\text{Count}[i]$.
 $\Theta(n^2)$ · (distribution counting $\Theta(n+\text{range})$)

7. Coin-Row DP · Dynamic Prog.

buffet, no two adjacent dishes
Max value from a row of coins, no two adjacent picked.

$F(i)=\max(F(i-1), c_i+F(i-2))$, $F(0)=0$, $F(1)=c_1$. Recover: if $F(i) \neq F(i-1)$ coin i taken $\rightarrow$ jump back 2.
 $\Theta(n)$

8. Change-Making DP · Dynamic Prog.

fewest jug pours to a line
Fewest coins to make amount m (unlimited coins).
 $F(m)=\min \text{ over } c_i \leq m \text{ of } F(m-c_i)+1$, $F(0)=0$. $+1$ = the last coin; $m-c_i$ = amount before it. Greedy fails on non-canonical sets (e.g. $\{1,3,4\}$).
 $\Theta(n \cdot \text{amount})$ pseudo-poly

9. Assignment · Branch & Bound · Backtracking

wedding seating + a shortcut
Assign n people $\rightarrow n$ jobs, min total cost; beat $n!$ by pruning.
Bound = committed cost + each unassigned person's cheapest **free** job (may double-use cols $\rightarrow$ optimistic). **Branch** person-by-person into free jobs.
Expand smallest bound; **prune** if bound $\geq$ incumbent.
exponential worst, prunes in practice

10. Dijkstra · Greedy

ripples spreading in a pond
Shortest path from source to all vertices (no negative weights).
① $\text{dist}(s)=0$, rest ∞ , keep $\text{prev}[]$ ② pick smallest-dist unvisited (now final) ③ relax: if $\text{dist}(u)+w < \text{dist}(v)$ update ④ repeat.
 $\Theta(E \log V)$ · no negative edges

11. Huffman Coding · Greedy

merge the two smallest teams
Optimal prefix code: frequent chars short, rare chars long.
① min-heap of leaf freqs ② extract 2 smallest, $\text{parent}=\text{sum}$ (left 0 / right 1), reinsert ③ until 1 root. Code = root $\rightarrow$ leaf path. Bits = $\sum \text{freq} \times \text{len}$.
 $\Theta(n \log n)$

12. 0/1 Knapsack DP · Dynamic Prog.

pack suitcase one item at a time
Max value of a subset within capacity W (item whole or left).
 $F[i][c]=\max(F[i-1][c], v_i+F[i-1][c-w_i])$ if $w_i \leq c$. Row 0 & col 0 = 0. Recover: $F[i][c] \neq F[i-1][c] \Rightarrow$ item i taken, drop w_i .
 $\Theta(nW)$ pseudo-poly

13. Floyd & Warshall · Dynamic Prog.

open hubs one at a time
All-pairs via intermediates numbered $1..k$ (k outermost loop).
Floyd (distances): $D[i][j]=\min(D[i][j], D[i][k]+D[k][j])$. **Warshall** (reachability): $R[i][j]=R[i][j]$ OR $(R[i][k] \text{ AND } R[k][j])$.
 $\Theta(n^3)$ time · $\Theta(n^2)$ space

Complexity reference

Algorithm	Best	Avg	Worst	Space
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$
Topological sort	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V+E)$	$\Theta(V)$
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$
Heapify (build)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
Comparison counting	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$
Distribution counting	$\Theta(n+r)$	$\Theta(n+r)$	$\Theta(n+r)$	$\Theta(n)$
Horspool/Boyer-Moore	$\Theta(n/m)$	$\Theta(n)$	$\Theta(nm)$	$\Theta(\text{alpha})$
Hashing	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Warshall / Floyd	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^2)$
0/1 Knapsack (DP)	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$
Coin-row / change	$\Theta(n) / \Theta(n \cdot \text{amt})$			$\Theta(n)$
Prim's MST (heap)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(V)$
Kruskal's MST	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(E \log E)$	$\Theta(V)$
Dijkstra (heap)	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(E \log V)$	$\Theta(V)$
Huffman	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$

$\Theta(nW)$ is **pseudo-polynomial** — W is a value, not the input size.